IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 1 October 2024, accepted 19 November 2024, date of publication 2 December 2024,
date of current version 23 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3509987

RESEARCH ARTICLE

MAGECODE: Machine-Generated Code Detection Method Using Large Language Models

HUNG PHAM[1], HUYEN HA[1], VAN TONG[1], DUNG HOANG[1], DUC TRAN[1], AND TUYEN NGOC LE[2,3]

[1]School of Information and Communication Technology, Hanoi University of Science and Technology, Hanoi 100000, Vietnam
[2]Department of Electronic Engineering, Ming Chi University of Technology, Taipei 24301, Taiwan
[3]Center for Reliability Engineering, Ming Chi University of Technology, Taipei 24301, Taiwan

Corresponding authors: Duc Tran (ductq@soict.hust.edu.vn) and Tuyen Ngoc Le (tuyennl@mail.mcut.edu.tw)

This work was supported in part by the National Science and Technology Council, Taiwan, under Grant NSTC 112-2222-E-131-003.

ABSTRACT The widespread use of virtual assistants (e.g., GPT4 and Gemini, etc.) by students in their academic assignments raises concerns about academic integrity. Consequently, various machine-generated text (MGT) detection methods, developed from metric-based and model-based approaches, were proposed and shown to be highly effective. The model-based MGT methods often encounter difficulties when dealing with source code due to disparities in semantics compared to natural languages. Meanwhile, the efficacy of metric-based MGT methods on source code has not been investigated. Moreover, the challenge of identifying machine-generated codes (MGC) has received less attention, and existing solutions demonstrate low accuracy and high false positive rates across diverse human-written codes. In this paper, we take into account both semantic features extracted from Large Language Models (LLMs) and the applicability of metrics (e.g., Log-Likelihood, Rank, Log-rank, etc.) for source code analysis. Concretely, we propose MageCode, a novel method for identifying machine-generated codes. MageCode utilizes the pre-trained model CodeT5+ to extract semantic features from source code inputs and incorporates metric-based techniques to enhance accuracy. In order to assess the proposed method, we introduce a new dataset comprising more than 45,000 code solutions generated by LLMs for programming problems. The solutions for these programming problems which were obtained from three advanced LLMs (GPT4, Gemini, and Code-bison-32k), were written in Python, Java, and C++. The evaluation of MageCode on this dataset demonstrates superior performance compared to existing baselines, achieving up to 98.46% accuracy while maintaining a low false positive rate of less than 1%.

INDEX TERMS Machine-generated code detection, large language model, metrics, CodeT5+.

I. INTRODUCTION

The emergence of advanced large language models (LLMs), such as ChatGPT and Gemini, has had a significant impact on all industries, including software development and programming education. Their proficiency in comprehending programming problem descriptions and generating code makes them valuable tools for enhancing student learning, thereby resulting in the widespread use of these artificial intelligence assistants. According to a recent survey by Das et al. [1], nearly 80% of students surveyed use ChatGPT to do their homework. Similarly, a recent survey by Haindl et al. [2] at St. Pölten University of Applied Sciences found that over 60% of students surveyed use virtual assistants like ChatGPT for their programming homework. However, the potential for student misuse of these models raises concerns about academic integrity and educational quality. Many students abuse virtual assistants to complete their homework without understanding or reviewing the solutions. This will eventually affect student creativity and independent thinking, which lead to negative consequences for both students and the education system.

However, most previous studies focused on detecting LLM-generated ***text*** with various methods, which can be divided into two categories [3]: metric-based and model-based methods. While the effectiveness of model-based

The associate editor coordinating the review of this manuscript and approving it for publication was Yang Liu.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

methods when performed on source code has been demonstrated to be limited, the applicability of metric-based methods on source code remains unexplored. Meanwhile, existing machine-generated ***code*** detection methods, such as DetectGPT4Code [4] and AIGCode Detector [5], have not achieved high accuracy nor have they been evaluated in high-stakes scenarios, where a false positive rate below 1% is necessary to minimize the risk of incorrect code removal and ensure that appropriate actions are taken.

In this paper, we aim to bridge the gap by developing a novel effective method to detect machine-generated codes in educational environments. The research focuses on detecting code solutions for programming problems rather than source code in large-scale software projects. This paper first thoroughly evaluates the performance of six metric-based machine-generated text detection methods when applied to source code to find highly adaptable metrics in source code analysis. Subsequently, we propose a novel model-based detector that utilizes the pre-trained model CodeT5+ [6] to extract semantic features from source code inputs and combines them with appropriate statistical metrics to detect machine-generated code. Extensive experiments are conducted to assess its effectiveness and compare it to current baseline methods.

To the best of our knowledge, there was no public dataset available for our experiments at the time of writing. Therefore, we constructed a new dataset containing both human-written and machine-generated source code, which are solutions for a set of programming problems. Machine-generated codes were obtained by querying three popular LLMs in code generation, namely ChatGPT [7], Gemini [8], and Code-bison-32k [9] with the descriptions of programming problems. The response codes from these LLMs were then tested against test cases to ensure their quality before being included in the final dataset. The final dataset contains more than 45,000 machine-generated code snippets written in three popular programming languages in education environments: Python, Java, and C++. Our experiments were then conducted on this newly constructed dataset.

To summarize, our work provides the following contributions:

- Introducing a new dataset for the machine-generated code detection problem, including over 45,000 source code produced by three well-known large language models: GPT-4-turbo, Gemini-pro-1.0, and Code-bison-32k. The dataset consists of examples of human-written and machine-generated source code in three programming languages: Python, Java, and C++. The dataset was published to Hugging Face[1] for the research community.
- Evaluating the effectiveness of metric-based detection methods commonly used for detecting machine-generated text across three code datasets of Python, Java, and C++.
- Proposing a novel machine-generated code detection method that utilizes the pre-trained encoder-only CodeT5+ model and integrated with highly applicable statistical metrics in source code analysis.

The remainder of this paper is organized as follows: Section II provides an overview of leading LLMs employed in code generation and explores existing detection methods, including those for identifying machine-generated text in general and specifically those designed for detecting machine-generated code. Section III discusses in detail our proposed method. The dataset construction and specification are described in Section IV. Section V presents the experimental results. The paper concludes with Section VI, which summarizes our findings.

II. BACKGROUND AND RELATED WORKS

A. ADVANCED LARGE LANGUAGE MODELS

This section explores the capabilities in code generation tasks of different Large Language Models (LLMs): GPT4, Gemini, and Code-bison-32k.

GPT4 [7] is a large multimodal model that can accept image and text inputs and produce text outputs. It is built upon Transfomer architecture to predict the next token in a text sequence. GPT4 outperforms both previous large language models and most state-of-the-art (SOTA) systems on a suite of traditional NLP benchmarks [7]. The performance of GPT4 in HumanEval benchmark achieves 67% compared to 65.8% of the current SOTA.

Gemini [8] is a family of multimodal models trained with diverse input to develop strong generalist abilities across multiple modals. A notable feature of Gemini is its ability to generate code, interpret user inputs describing desired functionalities, and translate them into functional code. This capability has significant potential to streamline software development workflows and enhance human-AI collaboration.

Code-bison-32k [9] is a specialized generative AI model focused on code generation and software development tasks. It excels in writing, debugging, and optimizing code across various programming languages. According to Google Deepmind, Code-bison-32k supports a wide range of coding languages, including C, C++, C#, Python, Java, JavaScript, and more than 30 additional languages [10].

The three aforementioned models were employed to produce machine-generated codes in our dataset. For GPT4, we used GPT4-turbo, which, during the experiments, points to the *gpt-4-0125-preview* model. For Gemini, we used *Gemini-1.0-pro* edition. Both GPT4 and Gemini are widely popular AI-powered products that have drawn significant attention from the community. On the other hand, Code-bison-32k is just a deep learning model itself and does not attract much attention. Nevertheless, Code-bison-32k is specifically designed for code-related tasks, whereas GPT4 and Gemini provide broader capabilities across different domains. Therefore, the inclusion of Code-bison-32k

[1] https://huggingface.co/datasets/HungPhamBKCS/magecode-dataset

enhances the variety and thoroughness of our dataset. It is also worth mentioning that we were aware of Github Copilot, which is also a widely used AI assistant for automated code production. However, as far as we know, this tool does not provide an API set for convenient code retrieval. After consideration, we decided to use only the three models mentioned earlier.

B. CODET5+ MODELS AND THE TRANSFORMER ARCHITECTURE

CodeT5+ [6] is a new family of open code large language models for code understanding and generation tasks. Developed by Salesforce AI Research, CodeT5+ models are aimed at overcoming two major constraints of existing code LLMs: architecture flexibility and restricted collection of pretraining tasks. Current code LLMs can only work in certain designs, like encoder-only or decoder-only. CodeT5+ models, on the other hand, have a flexible architecture that can work in encoder-only, decoder-only, or combined encoder-decoder modes to dynamically adapt to a wide range of downstream applications. Additionally, CodeT5+ models include a variety of pre-training tasks, such as span denoising, causal language modeling (CLM), contrastive learning, and text-code matching tasks, to fix the problem of limited pre-training tasks. This allows CodeT5+ models to bridge the gap between the pre-training and fine-tuning stages, as well as surpass existing code LLMs in aligning with the complexities of different downstream code tasks.

At the heart of CodeT5+ models lies the Transformer architecture, created by Google to handle natural language processing tasks. The Transformer architecture [11] was developed to tackle the challenges faced by Recurrent Neural Networks (RNNs) in capturing relationships between distant words in a phrase and their slow training speed. The Transformer architecture follows the *encoder-decoder* architecture of RNNs, where the encoder encodes an input sequence of symbols (which represents, for example, words in a sentence) to a sequence of continuous representation, and the decoder produces a sequence of symbols from the continuous representation. However, instead of using the recurrence mechanism in RNNs, both the Transformer encoder and decoder employ the *Multi-head self-attention* mechanism, which allows the models to understand global dependencies between words in a phrase or sentence. A *self-attention* function takes an input embedding $X = [x_1, x_2, \ldots, x_N] \in \mathbb{R}^{D\times N}$, each embedding x_i of dimension $D \times 1$ represents a word or token in the input sentence. Then, three following quantities, named *Value*, *Key*, and *Query*, are given as

$$V[X] = W_v X, \quad K[X] = W_k X, \quad Q[X] = W_q X \tag{1}$$

where W_v, W_k, and W_q represent weights for *Value*, *Key*, and *Query*, respectively. The terms *Query* and *Key* are derived from the field of information retrieval, where *Quey* is used to measure how much *attention* should be placed on different parts of the input sentence, and *Key* is used to compute the similarity between the *Query* and other parts in the sequence [12]. Combining them together, the *Attention* weight is as follows.

$$A[X] = \text{Softmax}\left(\frac{Q[X]K[X]^T}{\sqrt{d_k}}\right) \tag{2}$$

where d_k is the number of rows in W_k and W_q. *Value* $V[X]$ is the matrix that contains the actual information to be passed to the next layer in the model. From $V[X]$ and Attention weights $A[X]$, the output of a self-attention $\text{SA}[X]$ is computed as

$$\text{SA}[X] = V[X]A[X] \tag{3}$$

In the *Multi-head self-attention* mechanism, the self-attention function is performed H times with H different sets of W_v, W_k, and W_q, which outputs H self-attention outputs $\text{SA}_1[X], \text{SA}_2[X], \ldots, \text{SA}_H[X]$. Finally, these self-attention outputs are (vertically) *concatenated* and transformed linearly to produce the final output, i.e.,

$$\mathit{MultiHead}(X) = W_h[\text{SA}_1[X], \text{SA}_1[X], \ldots, \text{SA}_H[X]] \tag{4}$$

where W_h represents weights for multi-head computation.

The *Multi-head self-attention* mechanism was found to be beneficial and help self-attention perform well [12]. Both the Transformer encoder and decoder consist of multiple identical layers, each layer contains a Multi-head self-attention block and other transformations. Although only being a component in the whole architecture, this mechanism plays an important role in the success of Transformers.

By using Transformers as a foundation and taking an innovative approach to pre-training tasks as well as flexible architecture, CodeT5+ models have achieved state-of-the-art results across different code-related tasks, such as code comprehension and generation [6]. Therefore, we chose CodeT5+ models to be the base model for our MageCode method. The use of CodeT5+ in the method is detailed in Section III.

C. RELATED WORKS

1) MACHINE-GENERATED TEXT DETECTION METHODS

Recently, the problem of machine-generated text identification has drawn a lot of attention and research efforts. Various methods have been proposed to differentiate text produced by LLMs from text written by humans. These methods can be broadly categorized into metric-based and model-based methods [3].

In general, metric-based methods rely on pre-trained LLMs to analyze the text and extract statistical features from it, for example, the token-wise log probability or rank of each word in a document given its previous context [3]. This paper assesses six metric-based detection methods, including Log-Likelihood [13], Rank [14], Log-Rank [4], Entropy [14], GLTR [14], and LRR [15]. These metrics have been demonstrated in prior studies [3] to be rather effective in identifying machine-generated content from six LLMs, including ChatGLM [16], Dolly [17], ChatGPT-turbo [7],

GPT4All [18], StableLM [19], and Claude [20], across three distinct datasets.

In terms of model-based approaches, a classification model is developed by fine-tuning pre-trained language models with datasets that comprise both machine-generated and human-written texts [3], [13]. After the fine-tuning process, the developed classification model should be able to distinguish machine-generated content in the provided dataset. For example, OpenAIDetector [13] fine-tuned a RoBERTa model with texts produced by the largest GPT2 model. Similarly, ChatGPT Detector [21] was developed to detect ChatGPT-generated content by fine-tuning a RoBERTa model with the HC3 dataset as input. These methods have been demonstrated to achieve high accuracy in classifying text origins, even in challenging situations such as maintaining a false positive rate (FPR) below 0.1%. However, recent studies indicate that LLM-generated text detection methods are less effective when performed on source code data [22].

2) MACHINE-GENERATED CODE DETECTION METHODS

Compared to the machine-generated text detection problem, the task of identifying machine-generated code has garnered less attention and research efforts. Existing methods mainly rely on feature analysis, i.e., metric-based methods, notably AIGCode based on perplexity and DetectGPT4Code based on probability curvature.

AIGCode [5] is designed specifically to identify AI-generated code and prevent the misuse of LLMs among students in programming education. This method leverages targeted masking and a fine-tuned CodeBERT [23] model. By masking areas of the code with higher perplexity, it creates subtle variations that reveal patterns characteristic of AI-generated code. AIGCode then evaluates these variations using a scoring system that considers overall perplexity, variation in code line perplexity, and burstiness. Higher scores indicate a higher likelihood of AI generation, based on the observation that AI-generated code often exhibits lower perplexity and is less susceptible to perturbations. However, AIGCode was assessed using a limited dataset of approximately 5,000 machine-generated codes and only attained accuracy up to 92% for the three considered programming languages in this paper.

DetectGPT4Code [22] is a training-free method for detecting code generated by black-box models such as ChatGPT. Built upon the principles of the oringial zero-shot text detection method, DetectGPT [4], this method utilizes a small code language model as a surrogate white-box model to estimate the probability curve of the rightmost token in the candidate code. Particularly, DetectGPT4Code employs a smaller, open-sourced language model (LM) like Incoder-6B to calculate the probability curve of candidate code as well as generate multiple perturbed versions. Subsequently, various surrogate models ranging from 100M to 10B parameters, including PyCodeGPT-110M, PolyCoder, and CodeParrot, are utilized to estimate the probabilities of the ending tokens in both original and perturbed codes. Finally, the resulting DetectGPT4Code score is calculated to identify code snippets generated by language models.

III. MAGECODE

A. METHODOLOGY OVERVIEW

We developed MageCode, a machine-generated code detection method that utilizes the pre-trained model CodeT5+ 220M and integrates appropriate metrics. MageCode processes a code snippet input in four phases: Tokenization, Feature Extraction, Metric Calculation, and Classification.

In the Tokenization phase, the source code input is converted to embedding vectors by using CodeT5+'s tokenizer. Subsequently, the embedding vector is passed through the Encoder layers of the CodeT5+ model in the Feature Extraction phase to extract semantic features. In parallel, the Metric Calculation phase calculates several metrics which represent the statistical features of the source code. Then, statistical features are combined with semantic features to form a final feature vector. Finally, in the Classification phase, the classification layer consisting of two fully connected neuron network layers will employ the feature vector to perform the binary classification operation. The processing flow of MageCode is depicted in Figure 1. Details of each phase in the method are described in the following sections.

B. SOUCE CODE TOKENIZATION

The Tokenization phase is performed by the tokenizer of the CodeT5+ model. The tokenizer first breaks the input source code into smaller units called tokens. Since the tokenizer has a maximum input sequence length of 512 tokens, it either truncates or pads the source code input depending on its length. If the input source code exceeds the 512-token limit, the tokenizer will remove excess tokens from the end to ensure it fits within the maximum limit. On the contrary, if the source code is shorter than the limit, the tokenizer appends a special *[PAD]* token to the end until the sequence reaches the required length. The obtained embedding vectors will be used as input for the pre-trained encoder-only CodeT5+ model for feature extraction.

C. FEATURE EXTRACTION

In the Feature Extraction phase, the embedding vector obtained from the Tokenization phase is fed into the pre-trained encoder-only CodeT5+ model. Opting for an encoder-only model over a decoder-only or combined encoder-decoder model is justified by the findings that decoder-only models are often not ideal for understanding tasks such as retrieval and detection tasks compared to encoder-only models [6], and encoder-decoder models fail to beat state-of-the-art (SoTA) encoder-only or decoder-only baselines on retrieval and code completion tasks respectively [6]. The pre-trained encoder-only CodeT5+ model uses a multi-layer bidirectional Transformer-based architecture. The architecture is made up of 12 identical encoder

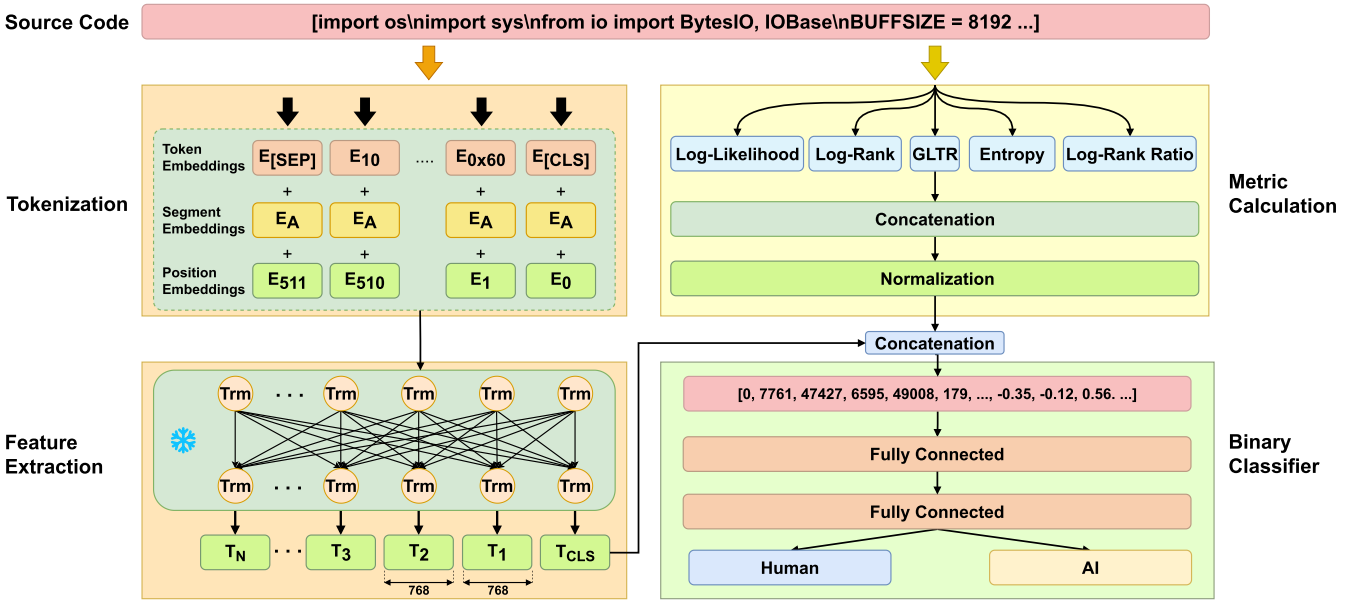


FIGURE 1. The overall architecture of MageCode.

block, and each block has two main parts: a bidirectional self-attention layer and a fully connected feed-forward neural network layer with ReLU activation [6].

The multihead self-attention mechanism and a normalization layer are applied to input embeddings X in each encoder block to make the initial output embedding Y.

$$Y = LayerNorm(X + MultiHead(X)) \tag{5}$$

Then, the output embedding Y is passed through a fully-connected feed-forward neural (FFN) network layer with ReLU activation function. Subsequently, the result of this transformation applies a normalization layer to produce the final output embedding Z.

$$Z = LayerNorm(Y + FFN(Y)) \tag{6}$$

The output embedding Z of an encoder block serves as the input embedding X for the successive encoder block. After 12 encoder blocks, we obtain the final hidden state of classification vector $[CLS]$, denoted as r ($r \in \mathbb{R}^{1\times 768}$), which is the aggregated representation of source code features.

D. METRIC CALCULATION

In parallel with the Tokenization and Feature Extraction phase, the source code input is used to calculate several metrics that represent different statistical aspects of the source code. This research considers the evaluation of the six following metrics.

Log-Likelihood [13] employs a language model to calculate the logarithmic probability of tokens. In particular, it computes the mean logarithmic likelihood of each word token in a given text to produce a score. A larger score suggests a higher likelihood that the text was produced by a machine. The Log-Likelihood score is computed as

$$\text{Log-Likelihood} = \frac{1}{t}\sum_{i=1}^{t} \log p_\theta(x_i|x_{<i}) \tag{7}$$

where t denotes the number of tokens, x_i represents the ith token in the given sequence of tokens, $x_{<i}$ refers to all tokens before i^{th} token, and $p_\theta(x_i|x_{<i})$ represents the conditional probability of observing token x_i given the context $x_{<i}$.

Rank [14] first computes the absolute rank of each word in a text based on its preceding context. Following that, the rank score of the given text is determined by calculating the average of these rank values across all words. A lower score indicates a higher probability of the text produced by a machine. Rank score is calculated as

$$\text{Rank} = \frac{1}{t}\sum_{i=1}^{t} r_\theta(x_i|x_{<i}) \tag{8}$$

where $r_\theta(x_i|x_{<i}) >= 1$ is the rank of token x_i conditioned on the previous tokens.

Log-Rank [4], different from the Rank metric, calculates the score by applying the logarithm function to the rank value of each word instead of directly utilizing the absolute rank. The Log-Rank can be defined as

$$\text{Log-Rank} = \frac{1}{t}\sum_{i=1}^{t} \log r_\theta(x_i|x_{<i}) \tag{9}$$

In a similar manner to the Rank score, Entropy [14] computes the score of a text by averaging the entropy value of each word based on its prior context. Previous research [4], [14] points out that texts produced by machines are more likely to have lower Entropy scores. The Entropy score is as

follows

$$\text{Entropy} = -\frac{1}{t}\sum_{i=1}^{t} p_\theta(x_i|x_{<i}) \log p_\theta(x_i|x_{<i}) \tag{10}$$

GLTR [14] is specifically developed as a supplementary tool to assist in identifying whether a given text is machine-generated. This work employs the GLTR method incorporated into the MGTBench Framework [3], with a specific focus on Test-2 features. These features encompass the assessment of the proportion of words that rank within the top 10, 100, 1,000, and other categories.

Log-Likelihood Log-Rank Ratio (LRR [15]) is proposed by Su et al. that combines Log-Likelihood and Log-Rank to offer comprehensive insights for a given text. The LRR can be computed as

$$\text{LRR} = \frac{\text{Log-Likelihood}}{\text{Log-Rank}} = \frac{\sum_{i=1}^{t} \log p_\theta(x_i|x_{<i})}{\sum_{i=1}^{t} \log r_\theta(x_i|x_{<i})} \tag{11}$$

After being calculated, normalization is applied to shape these metric results into the uniform distribution. Finally, the normalized metrics are appended to the end of r to create the final feature vector. This feature vector serves as the input to the classification layer in the binary classification phase. It should be noted that not all six metrics presented above are integrated in MageCode. Rather, the suitability of each metric will be investigated (Section V-A), and only metrics that show promising results will be included in the final feature vector.

E. BINARY CLASSIFICATION

The classification layer consists of two fully-connected feed-forward neural (FFN) network layers, with a ReLU activation function positioned between them. The first FFN network layer contains 1024 neurons, while the second layer has only 1 neuron. Following the second layer, the sigmoid function is applied to the single output neuron to carry out the binary classification operation, which effectively classifies the input source code as either human-written or machine-generated.

F. TIME COMPLEXITY

The proposed technique includes two primary components: feature extraction utilizing the pre-trained model CodeT5+ and metric calculation. The time complexity for metric calculation is $O(n)$, with n representing the length of the input sequence. Additionally, CodeT5+ comprises Transformer blocks that incorporate self-attention methods. The attention mechanism, developed by Google, has a time complexity of $O(n^2 \times d)$, where n represents the sequence length and d is the representation dimension [11]. Consequently, the time complexity of the proposed approach is represented in Equation 12.

$$O(n^2 \times d) + O(n) = O(n^2 \times d); \tag{12}$$

```python
def segments_count(a, b):
    segments = [6, 2, 5, 5, 4, 5, 6, 3, 7, 6]  # 0-9 segment counts
    total_segments = 0
    for number in range(a, b + 1):
        while number:
            total_segments += segments[number % 10]
            number //= 10
    return total_segments

a, b = map(int, input().split())
print(segments_count(a, b))
```

LISTING 1. Python code snippet example.

G. EXAMPLE

We provide a specific example to demonstrate the detection flow of MageCode. Suppose we have a Python code snippet as follows:

First, the code is tokenized using the tokenizer of CodeT5+ model. The output is an embedding vector with 512 dimensions:

$$\begin{bmatrix} 1 & 536 & 5155 & \dots & 0 & 0 \end{bmatrix}$$

This embedding vector then becomes the input of the pre-trained encoder-only CodeT5+ model, which uses the Transformer architecture for feature extraction. We obtain the final hidden state of the [CLS] vector, which has 768 dimensions, after passing the embedding vector through 12 encoder blocks:

$$\begin{bmatrix} -0.54 & -0.59 & -0.29 & \dots & -0.12 & -0.49 \end{bmatrix}$$

Subsequently, the code snippet input is used to calculate the aforementioned metrics. Those metrics are calculated by using the CodeBERT-base-mlm model [23] as the base model (section V-A). The calculated results are then concatenated to form a temporary vector, referenced as a "metric vector." Suppose all six metrics are integrated into the MageCode method, the obtained "metric vector", as shown in the equation at the bottom of the page.

The listed order follows the order in which six metrics are described in section III-D: Log-Likelihood, Rank, Log-Rank, Entropy, GLTR, and LRR. Since the GLTR method employed in this work encompasses the assessment of the proportion of words that rank within the top 10, 100, 1,000, and other categories, it has four outputs instead of one as remaining metrics. The "metric vector" is then normalized, with mean and standard deviation determined after the training process on the train split of dataset. The normalized "metric vector":

$$\begin{bmatrix} 0.85 & 1.16 & -0.10 & -1.19 & 1.36 & -1.21 & -0.79 & -0.49 & 0.76 \end{bmatrix}$$

is appended to the end of the feature vector calculated above to form the complete feature vector, as shown in the equation at the bottom of the next page.

$$\begin{bmatrix} -11.23 & 911.42 & 3.82 & 0.49 & 0.35 & 0.20 & 0.27 & 0.18 & -2.94 \end{bmatrix}$$

This final feature vector serves as the input to the fully-connected layer in the Classification phase, which classifies the provided code snippet as human-written or machine-generated.

IV. DATASET CONSTRUCTION

This section outlines our dataset construction process, which is illustrated in Figure 2. First, we collected programming problems and their solutions from previous studies. After applying some pre-processing steps, we queried advanced LLMs, such as GPT4, with the collected problems to produce machine-generated codes, while the original solutions were labeled as human-written codes. Finally, the entire collected dataset was divided into training, validation, and test sets, ensuring that there is no overlap among these sets, meaning no solutions in two different sets that solve the same programming problem.

A. CODING PROBLEM COLLECTION

We collected programming problems from two open-source datasets: Topics in Algorithmic Code Generation (TACO) [24] and CodeContest [25].

TACO [24] is a large-scale, publicly accessible dataset specifically curated to advance the development of cutting-edge code generation models. Focusing on algorithmic problems, this extensive dataset comprises 26,443 programming challenges and their corresponding 1,539,152 high-quality Python solutions. Designed to elevate training and evaluation benchmarks, TACO provides a robust foundation for advancing the capabilities of code generation systems.

CodeContests [25] is a dataset specifically created for training AI models to solve programming problems. It was introduced by Google DeepMind in 2022 for developing AI systems that can compete in coding contests. The dataset contains a variety of programming challenges from popular platforms like Aizu, AtCoder, CodeChef, Codeforces, and HackerEarth. Each programming problem includes test cases with both input and expected output, as well as examples of correct and incorrect solutions written in Python, Java, and C++. This comprehensive dataset was used in training AI models like AlphaCode, which have shown promising results in competitive programming.

The dataset used for this study consisted of Python, C++, and Java questions and solutions. Python questions and solutions were taken from the TACO dataset, while C++ and Java questions and solutions were taken from the CodeContest dataset. Only data from before January 1, 2022, was included to avoid the influence of advanced AI models like ChatGPT and Gemini. Additionally, answers from programming practice websites like GeeksForGeeks were excluded to prevent contamination from solutions generated by AI assistants. After being preprocessed, the dataset contained 3,499 Python questions with 757,794 solutions, 11,086 C++ questions with 1,756,180 solutions, and 10,118 Java questions with 900,079 solutions. All of these code samples were labeled as *human-written*.

Notwithstanding the aforementioned preprocessing phase, it is crucial to emphasize that there is no assurance that all ''human-written'' labeled solutions are entirely devoid of the influence of LLMs. This constitutes a constraint in our dataset development procedure. Nonetheless, given that these solutions are primarily sourced from competing programming platforms prior to the emergence of user-friendly and robust web-based LLMs like ChatGPT, the danger of LLM contamination in these solutions is comparatively little and appropriate for this study.

B. MACHINE-GENERATED CODE COLLECTION AND TESTING

To obtain machine-generated solutions, we queried three advanced LLMs with collected programming problems: GPT-4-turbo from OpenAI, Gemini-1.0-pro, and Code-bison-32k from Google. This work utilizes OpenAI's API service for GPT-4-turbo generation and Google AI's API service for Gemini-1.0-pro and Code-bison-32k generation. The decoding temperature is configured to 0.7. This is a critical parameter that balances the predictivity (with lower values) and creativity (with higher values) of generated content. The value of 0.7 allows multiple solutions to the same programming problem to not be too similar to each other while still making them pass as many tests as possible. It is also used by TACO [24] when experimenting with GPT4.. All other settings are maintained at their default values. The experiments were conducted between February 25 and June 18, 2024. All solutions were generated by prompting 3,499 Python questions, 11,086 C++ questions, and 10,118 Java questions into the three above-mentioned LLMs. The prompt used to collect machine-generated source code is detailed in Table 1. The prompt involved stating the question details and desired programming language, providing starter code if necessary, outlining the input format, and then requesting code generation from the queried LLM, without explanation, test cases, or example usage for generated source code. For each question, up to five solutions from each language model are collected.

We then evaluated the machine-generated solutions using test cases from the TACO and CodeContest datasets. To be included in the dataset, a solution had to pass at least one test case, reflecting the common practice among students who aim to earn partial points in programming assignments. Besides, selected solutions were not required to pass every

$$[-0.54 \quad -0.59 \quad -0.29 \quad \ldots \quad -0.12 \quad -0.49 \quad 0.85 \quad 1.16 \quad \ldots \quad -0.49 \quad 0.76]$$

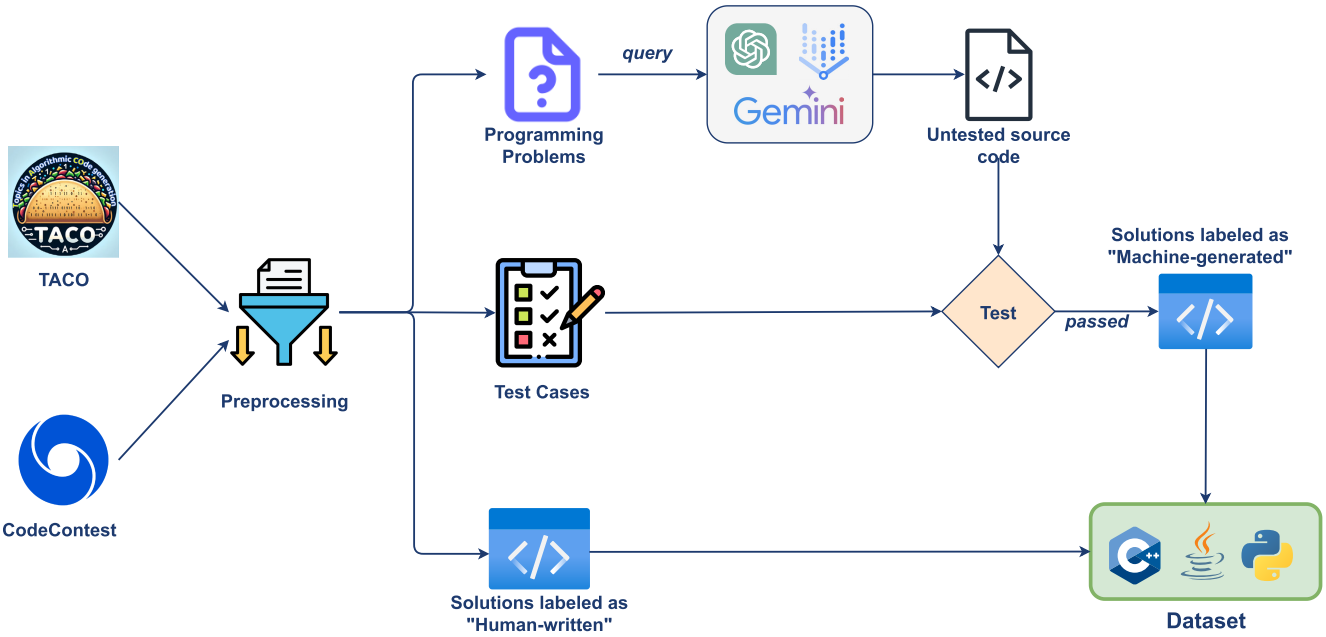


FIGURE 2. Dataset construction flow.

TABLE 1. The prompt used to query GPT-4-turbo, Gemini-1.0-Pro, and Code-bison-32k models.

Solve the coding problem below in <programming_language> programming language: <question> Use starter code below as template: <starter_code > Use <input format>. Only return code, don't explain anything, don't include test case or example usage

test case, as many students only submit code that is sufficient enough to help them pass their exams or homework. The collected human-written solutions above also include "incorrect" solutions that only succeed on a subset of test cases. Including these incomplete solutions in both human-written and machine-generated solution sets adds diversity to the dataset, enabling models trained on the dataset to learn a variety of solution patterns rather than just perfect ones. Moreover, obtaining machine-generated solutions that can pass all test cases of a programming problem is both costly and time-consuming since we would need to query LLMs several times until we received at least one perfect solution. That process is challenging, if not unfeasible, for medium and hard programming problems.

To automatically test machine-generated solutions from LLMs, we employed two approaches. For Python solutions, we utilized the existing source code provided by the TACO project for testing Python code. In the case of Java and C++ solutions, we use Judge0 [26], a robust and scalable online code execution system. As an open-source project with a readily available Docker image, Judge0 has become a crucial part of various production systems requiring online code execution capabilities. In our approach, we deployed a local Judge0 server and leveraged its APIs to interact with the server. Through those APIs, we send solutions generated by LLMs to the Judge0 server, along with a list of test cases and the desired programming language environment. The server then evaluates the submitted code in the specified language against the provided test cases. After the evaluation, the server returns a result indicating whether the solution is *passed* if at least one test case is successful, or *failed* if no test case is passed or if there are compilation or runtime errors. Only *passed* solutions are included in the final dataset. These solutions are labeled as *machine-generated*.

C. DATASET SPLIT AND SPECIFICATION

After the collection phase, we obtained over 81,000 human-written and over 45,000 machine-generated source code. An overview of the dataset is described in Table 2.

TABLE 2. Overview of the dataset used for detecting code generated by large language models.

Programming Language	Number of Code Samples	
	Human-written code	Machine-generated code
Python	16,507	17,438
C++	54,944	14,973
Java	10,118	12,612

The dataset is split into three smaller datasets, each of which corresponds to a programming language. Each split

dataset is then divided into the training set, the validation set, and the test set, with their respective proportions of 76%, 4%, and 20%. The data in the training set and validation set are evenly distributed between human-written and machine-generated categories. Conversely, the test set includes the remaining data, which preserves the original distribution of the dataset and serves as realistic data for the testing procedure. It is worth noting that each programming problem has several solutions, which may exhibit a high degree of similarity or possess identical patterns. Therefore, while splitting the dataset, we ensure that there are no two solutions from two separate sets that solve the same problem to avoid data leakage when training and testing our method. The number of samples in each set across different groups is described in Table 3.

TABLE 3. The number of samples in the training set, validation set, and test set across three programming languages.

	Training		Validation		Test	
	Human	Machine	Human	Machine	Human	Machine
Python	12,544	12,544	659	657	3,304	4,738
Java	7,689	7,686	404	403	2,025	4,523
C++	11,375	11,379	598	596	43,000	2,998

Each sample in the dataset possesses three properties: *task_id*, *code*, and *label*. The *task_id* field in the Python dataset represents the ordinal number of programming problems in the train split of the TACO dataset. For Java and C++ datasets, this field corresponds to the ordinal number of programming problems in the train split of the CodeContest dataset. The *code* field contains source code, and the *label* field indicates the origin of the source code, with 0 denoting human-written code and 1 denoting machine-generated code. The dataset is stored in several CSV files, each representing different sets and programming languages. It is publicly published on Hugging Face for the research community.

V. EVALUATION

This section presents evaluation results that assess the performance of MageCode and other detectors. Firstly, we evaluate several metric-based machine-generated *text* detection methods on source code input. The metrics that show promising results will be selected for integration into the MageCode method. In the following experiment, we evaluate the performance of MageCode and compare it to current baselines. Finally, we examine the potential effects of different factors on the performance of MageCode.

This study evaluates Accuracy, F1-score, and Area Under ROC Curve (AUROC) for performance measurement. Additionally, we also consider *True Positive Rate at a False Positive Rate* (TPR@aFPR) to measure the sensitivity of the method at very low FPR. This metric offers valuable information about the model performance in critical detection scenarios. Optimal true positive rate (TPR) while minimizing false positive rate (FPR) is crucial, as the primary risks in critical detection scenarios often arise from false positives, which refer to the erroneous identification of human-written code as machine-generated. This paper assesses detection methods under the range of $a \in \{10, 1, 0.1\}$.

All experiments are conducted on the newly constructed dataset as described in Section IV. As can be seen from Table 3, the C++ dataset has 43,000 human-written samples, which is 14 times greater than the number of machine-generated samples. In order to address the issue of imbalance during evaluations, we made the decision to reduce the size of the human-written set by only using human-written samples in a quantity equivalent to the machine-generated set, which consists of 2,998 samples.

The experiments are implemented in a Window 11 PC equipped with CPU Core i9 14900k, 128 GB RAM and GPU Nvidia RTX4070Ti.

A. EVALUATION OF METRIC-BASED METHODS

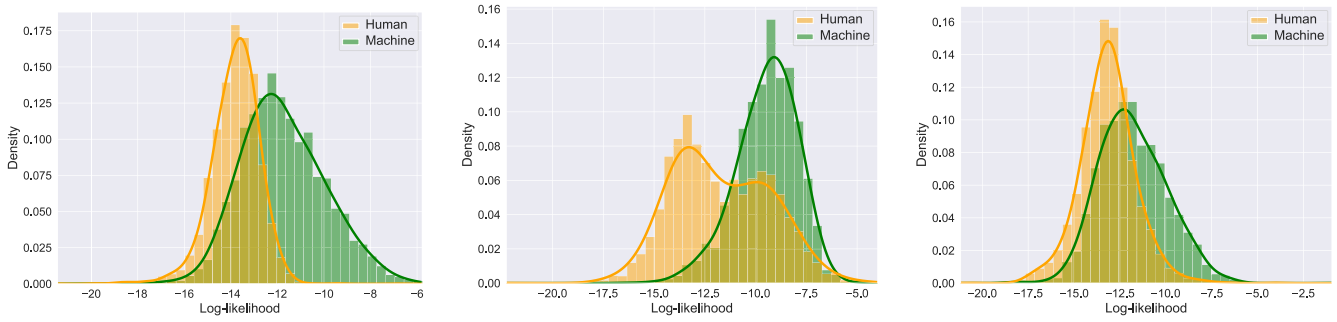
The six evaluated metric-based detection methods are Log-Likelihood [13], Rank [14], Log-Rank [4], Entropy [14], GLTR [14], and LRR [15]. For each method, this work uses the CodeBERT-base-mlm model [23] as the base model to extract logits from. From the metrics extracted using the CodeBERT-base-mlm model, a logistic regression model is constructed to provide concrete predictions. Table 4 presents the evaluation results across three programming languages.

Table 4 clearly demonstrates that the Log-Likelihood method surpasses other metrics in Python and C++ datasets, achieving the highest figures in Accuracy, F1-Score, and AUROC. The results demonstrate the excellent effectiveness of the Log-Likelihood metric in correctly identifying machine-generated code in these chosen programming languages. Moreover, the illustration in Figure 3 reveals a clear differentiation between positive and negative samples in the distribution of Log-Likelihood score. Machine-generated codes typically exhibit Log-Likelihood scores positioned to the right of the Log-Likelihood score distribution typically observed in human-written codes. This discovery in the source code corresponds to results obtained in natural language analysis [13], [14].

The performance of Entropy exhibits variability among different programming languages. In the Java programming language, it attains the highest values of Accuracy (76.51%), F1 score (82.61%), and AUROC (79.07%). However, the performance of this metric is comparatively inferior in Python and C++, particularly in C++, with Accuracy of 53.84%, F1 score of 55.90%, and AUROC of 55.32%. Remarkably, there exists a notable disparity in the entropy distribution between code created by machines and code written by humans for Java (Figure 6). However, the distributions for Python and C++ are relatively comparable, especially in the C++ dataset. This disparity explains why the use of entropy for classifying C++ code from both human-written and machine-generated sources is less efficient. Moreover, our experiments using source code input confirm the consistent finding that the entropy of machine-generated material is

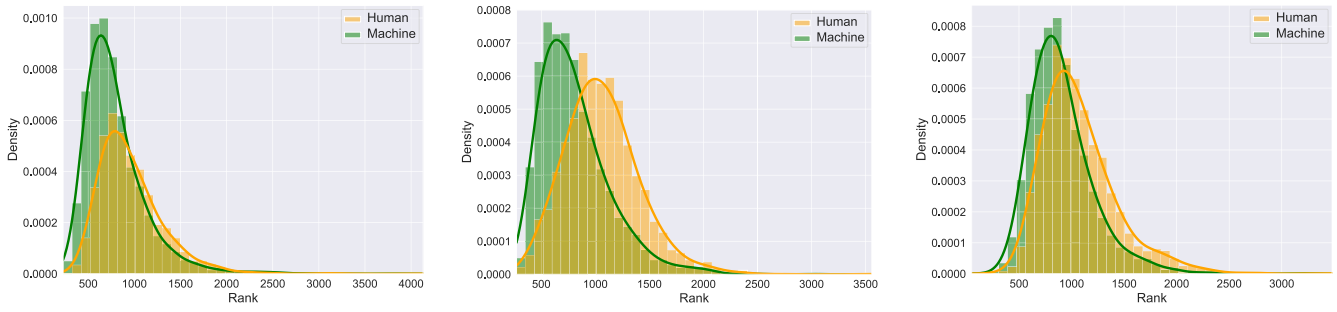
TABLE 4. Evaluation of metric-based machine-generated text detection methods across three programming languages. (Unit: %).

Metrics	Python			Java			C++		
	Accuracy	F1 Score	AUROC	Accuracy	F1 Score	AUROC	Accuracy	F1 Score	AUROC
Log-Likelihood	**77.44**	**79.43**	**84.77**	73.17	79.67	77.34	**66.24**	**64.96**	**71.91**
Rank	63.12	69.06	66.80	69.32	76.46	73.36	61.37	63.51	65.67
Log-Rank	72.43	73.94	78.66	74.07	80.48	77.81	62.33	60.40	67.23
Entropy	63.16	67.34	68.26	**76.51**	**82.61**	**79.07**	53.84	55.90	55.32
GLTR	73.30	74.71	78.71	73.31	79.84	77.51	62.69	60.53	67.04
LRR	54.38	60.05	55.23	54.72	62.96	55.40	55.90	56.08	57.84



(a) Python (b) Java (c) C++

FIGURE 3. Log-likelihood score distribution across three programming languages.



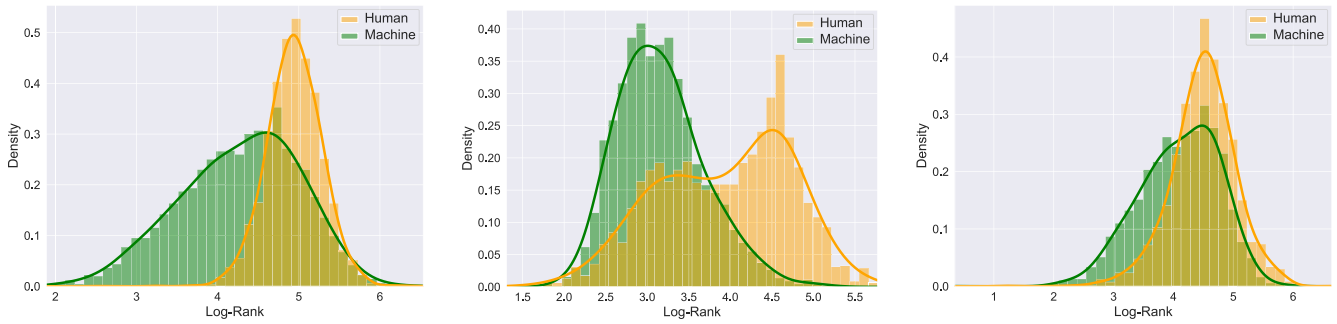
(a) Python (b) Java (c) C++

FIGURE 4. Rank score distribution across three programming languages.

generally lower than that of human-written information, as demonstrated in prior studies [4], [14].

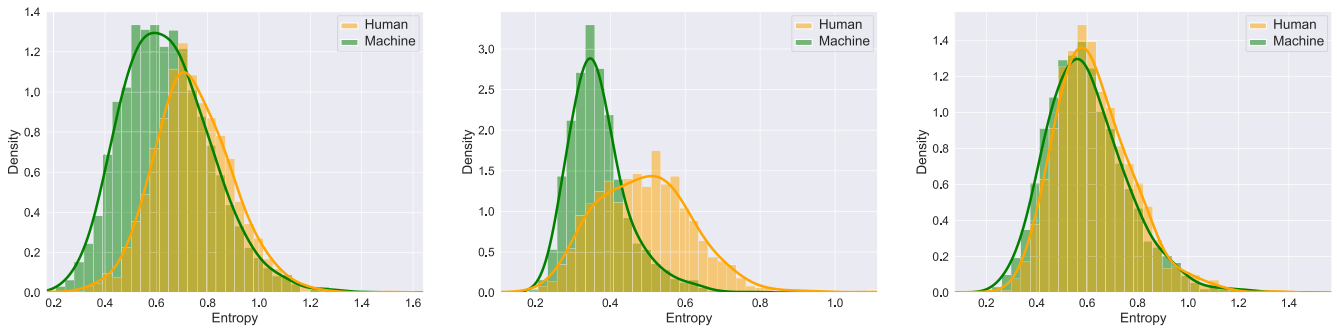
Comparing the performance of the Rank and Log-Rank methods on the datasets, Log-Rank consistently provides substantial enhancements. While demonstrating a similar performance compared to Rank in the C++ dataset, Log-Rank surpasses Rank in Python and Java datasets outright. Within the Java dataset, the Log-Rank algorithm attains an Accuracy of 74.07%, F1 Score of 80.48%, and AUROC of 77.81%. These results even exceeds the measurements of Log-Likelihood on the same dataset, and are only lower than the Entropy metric. The distribution of Rank and Log-Rank scores is shown in Figure 4 and Figure 5, respectively. Although both strategies take the observed rank of each token into account when making the prediction, the use of the logarithm transformation in Log-Rank improves its effectiveness as a detector in comparison to the Rank metric. Using the Java dataset as an illustration, the Rank distributions depicted in Figure 4 for machine-generated code and human-written code are difficult to distinguish due to their significant overlap. In contrast, the Log-Rank score distribution shown in Figure 5 offers a more distinct differentiation. Consistent with findings on natural language content [14], [27], we also note that machine-generated code generally exhibits lower average values for both Rank and Log-Rank scores.

GLTR shows solid performance across three distinct programming domains. On Python and C++ datasets, its performance is just lower than that of Log-Likelihood. For instance, the GLTR method achieves an Accuracy score of



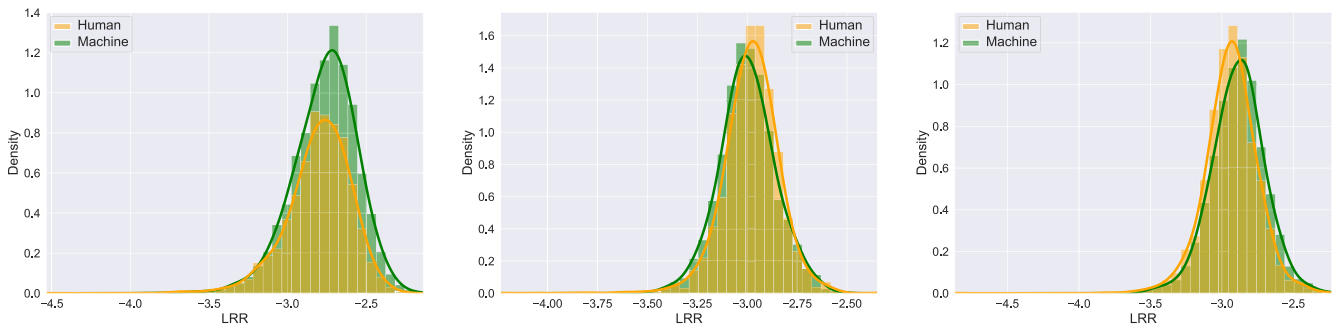
(a) Python (b) Java (c) C++

FIGURE 5. Log-rank score distribution across three programming languages.



(a) Python (b) Java (c) C++

FIGURE 6. Entropy score distribution across three programming languages.



(a) Python (b) Java (c) C++

FIGURE 7. LRR score distribution across three programming languages.

73.30%, an F1 Score of 74.71%, and an AUROC of 78.71% on the Python dataset. The stable performance of this metric across many languages can be ascribed to its consideration of the proportion of tokens that are ranked among the top 10, 100, 1,000, and so on.

The LRR method consistently demontrates the poorest performance in all measurements and languages, with results ranging from 54% to 62%. This performance indicates that, while Log-Likelihood and Log-Rank independently show promising outcomes, the combination of both approaches does not improve detection ability. Contrarily, it continuously exhibits poor performance in all programming languages. Figure 7 displays the distribution of LRR scores among three programming languages. The distribution of machine-generated code and human-generated code exhibits no significant distinction based on the LRR scores. This observation implies that LRR may not be a very efficient approach for identifying code created by an LLM.

Furthermore, we performed a Friedman test on the performance of six metric-based methods. The Friedman test is a non-parametric statistical test used to compare the performance of multiple models across multiple datasets

based on the average ranking of tested models. This test evaluates whether there are significant differences in the performance of tested models [28]. When performning the test, the level of significance is set to 0.05, which was used in the previous studies [28], [29]. AUROC is the selected metric for performance comparison and ranking. Since there are six metrics evaluated, the test was performed with 6 degrees of freedom.

TABLE 5. Rankings of metric-based machine-generated text detection methods across three programming languages.

Metric	Rankings			Average Ranking
	Python	Java	C++	
Log-Likelihood	1	4	1	2
Rank	5	5	4	4.67
Log-Rank	3	2	2	2.33
Entropy	4	1	6	3.67
GLTR	2	3	3	2.67
LRR	6	6	5	5.67

Table 5 shows the rankings of six metrics across three datasets in terms of performance. AUROC is selected for performance comparison and ranking. For each dataset, rankings are from 1 to 6, with rank 1 assigned to the best metric and rank 6 assgned to the worst metric. The average ranking of each metric is calculated as the average of its rankings across three datasets.

Based on these rankings, we can calculate the Friedman test statistic result, which is 8.90. With 6 degrees of freedom and the significance level of 0.05, the critical chi-squared value is 12.59, taken from the chi-square table. Since $8.90 < 12.59$, the null hypothesis of Friedman test cannot be rejected, which means there is no statistically significant difference in the performance of tested metrics.

There is a possibility that this result is caused by the performance of evaluated metrics on the Java dataset, where the ranks differ the most. If we exclude the rankings from the Java dataset and solely consider rankings from the two remaining datasets, the Friedman test would yield a result of $13.23 > 12.59$. This new result means *there is a statistically significant difference in the performance of six metrics on Python and C++ datasets.*

Additionally, from Table 5, it is simple to tell that Log-Likelihood, Log-Rank, and GLTR stand out from the remaining metrics in terms of performance. Besides, despite the inferior result on the Python and C++ datasets, the fact that Entropy is the metric that has the best ranking on the Java dataset makes it unignorable.

The evaluation results of metric-based methods clearly indicate that Log-Likelihood, Entropy and GLTR are the most applicable metrics in source code analysis. Rank and Log-Rank are essentially identical metrics, with a minor distinction in the calculation of the final score of a given text. Nevertheless, Log-Rank has been empirically demonstrated to be the better metric in identifying machine-generated source code when compared to Rank. LRR is without a doubt the metric with the worst performance. Following this findings, we have selected Log-Likelihood, Log-Rank, Entropy, and GLTR to be integrated into the MageCode method.

B. EVALUATION OF MAGECODE

The experiment in this section evaluates the MageCode method integrated with the four above selected metrics across three datasets. The evaluation results are subsequently compared to the existing baselines, namely OpenAI Detector [13] and DetectGPT4Code [21]. As introduced earlier, OpenAI Detector was developed by fine-tuning the RoBERTa model using the output from the GPT-2 1.5-billion-parameter model, which was originally used to detect text generated by GPT-2. Meanwhile, DetectGPT4Code is inspired by the DetectGPT method and introduces enhancements by utilize a surrogate model, PolyCode-160M, to overcome the difficulties encountered by DetectGPT when the source code model is a black box (e.g., GPT4).

Table 6 shows the results of evaluating the three mentioned methods in detecting machine-generated code based on metrics such as Accuracy, F1-Score, AUROC, TPR@0.1%FPR, TPR@1%FPR, and TPR@10%FPR.

We find that MageCode consistently exhibits superior performance across all assessed metrics and programming languages. It showcases exceptional performance in Python, with an Accuracy of 98.46% and F1 Score of 98.70%, AUROC of 99.83%, and TPR values of 86.87%, 96.14%, and 99.77% at FPRs of 0.1%, 1%, and 10% correspondingly. In contrast, the OpenAI Detector and DetectGPT4Code show a notable performance gap, with the OpenAI Detector specifically demonstrating a low Accuracy of 59.19% and insignificant TPR values at lower FPR thresholds. Similarly, the MageCode method for Java demonstrates strong performance with an Accuracy of 98.05%, F1 Score of 98.59%, AUROC of 99.47%, and TPR values of 37.89%, 85.45%, and 99.76% for FPR values of 0.1%, 1%, and 10% respectively. Both the OpenAI Detector and DetectGPT4Code exhibit poor performance in all measures, particularly in their failure to obtain significant TPR values at lower FPR levels. On the C++ dataset, MageCode consistently performs better than other methods, achieving an Accuracy of 95.53%, F1 Score of 75.46%, AUROC of 99.06%, and TPR values of 47.97%, 85.36%, and 97.93% at FPR values of 0.1%, 1%, and 10% correspondingly.

The main reason for the low accuracy of the OpenAI Detector model while processing source code inputs is its lack of training on source code, which hinders its ability to accurately distinguish between code written by humans and code generated by machines. On the other hand, DetectGPT4Code is a metric-based approach that utilizes the probability curvature metric. Due to the intrinsic complexity of source code in terms of syntax, structure, and logic, depending on a single metric threshold is inadequate for distinguishing between human-written and machine-generated code, as shown by prior studies [3]. To overcome the limitations of these approaches, MageCode has employed

TABLE 6. Comparison results of MageCode with current baselines across three programming languages. (Unit: %).

Python						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
MageCode	**98.46**	**98.70**	**99.83**	**86.87**	**96.14**	**99.77**
OpenAI Detector	59.19	73.58	39.65	0	0.06	1.65
DetectGPT4Code	58.92	74.15	39.56	0.17	1.31	8.91
Java						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
MageCode	**98.05**	**98.59**	**99.47**	**47.89**	**85.45**	**99.76**
OpenAI Detector	54.81	60.86	51.91	0	0.04	3.52
DetectGPT4Code	69.08	81.71	51.92	0.20	0.84	9.02
C++						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
MageCode	**95.53**	**95.46**	**99.06**	**47.97**	**85.36**	**97.93**
OpenAI Detector	61.74	69.04	54.86	0.03	0.13	4.04
DetectGPT4Code	50.00	66.67	34.79	0	0.23	3.84

the CodeT5+ model, which is highly proficient in extracting significant characteristics from source code. Additionally, it has integrated multiple metrics to accurately categorize the source code.

C. ABLATION STUDIES

This section conducts three ablation experiments to investigate the effects of different factors on MageCode: the impact of pre-trained Code LLM, the impact of integrated metrics, and the impact of token length.

1) INFLUENCE OF PRE-TRAINED CODE LLM

Table 7 presents the comparison results when applying different base LLM models in the tokenization and feature extraction phases of the MageCode method. Three base LLM models considered in this experiment are CodeT5+ [6], CodeBERT [23], and PolyCoder [30].

It can be observed that, across all programming languages, CodeT5+ consistently demonstrates competitive or superior results compared to CodeBERT and PolyCoder. In the Python dataset, despite better performance of CodeBERT in TPR@0.1%FPR and TPR@1%FPR, CodeT5+ is still the top performer by achieving the highest results in Accuracy (98.46%), F1-Score (98.70%), AUROC (99.83%). CodeBERT closely follows with Accuracy of 98.29%, F1-Score of 98.54%, and AUROC of 99.81%. The difference in these metrics between CodeT5+ and CodeBERT is under 0.2%, which proves CodeBERT as a competitive candidate compared to CodeT5+. However, in Java and C++ datasets, CodeT5+ demonstrates superior results by achieving the highest results in all measurements, with Accuracy, F1-Score, and AUROC over 98% in Java and over 95% in C++. Meanwhile, CodeBERT shows significantly worse results, with 2-3% worse in Accuracy, F1-Score, and AUROC compared to CodeT5+. Although demonstrating promising results in TPR@a%FPR in the Python dataset, the performance of CodeBERT in these metrics is inferior when applied to Java and C++. On the other hand, PolyCoder is the worst model by consistently exhibiting inferior results compared to the other two models.

We observe that CodeBERT exhibits a notable decrease in performance when applied to C++ in comparison to Python and Java. This disparity can be ascribed to the lack of model pre-training on C++. On the other hand, the inadequate performance of PolyCoder on all three datasets may be attributed mostly to its decoder-only Transformer-based architecture. This architecture is better suited for text production tasks but less efficient for classification tasks when compared to encoder-only models such as CodeT5+ and CodeBERT.

2) INFLUENCE OF INTEGRATED METRICS IN MAGECODE

This section examines the influence of integrated metrics on the performance of the MageCode method. As presented in Section III, source code features extracted from the encoder-only CodeT5+ model and metrics calculated from source code input are combined to create the final feature vector before heading to the classification phase. The effectiveness of these metrics when being applied independently in detecting machine-generated code has been thoroughly studied in Section V-A. To determine how these metrics affect MageCode when integrated together into the feature vector, we re-trained and evaluated MageCode in two scenarios: *Metric Integrated*, which means the feature vector contains both CodeT5+ extracted features and calculated metrics, and *CodeT5+ Only*, where the feature vector only contains CodeT5+ features. Table 8 shows the evaluation results of these scenarios.

The results in Table 8 reveal that two scenarios demonstrate highly competitive results, with a slightly better performance of *Metric Integrated* overall. The *Metric Integrated* approach consistently outperforms *CodeT5+ Only* in Accuracy,

TABLE 7. Performance metrics of various base models in detecting machine-generated source code. (Unit: %).

Python						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
CodeT5+	**98.46**	**98.70**	**99.83**	86.87	96.14	**99.77**
CodeBERT	98.29	98.54	99.81	**93.14**	**97.30**	99.60
PolyCoder	86.93	88.81	94.84	29.36	54.58	84.70
Java						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
CodeT5+	**98.05**	**98.59**	**99.47**	**47.89**	**85.45**	**99.76**
CodeBERT	95.42	96.71	98.25	10.59	59.56	97.66
PolyCoder	84.85	89.60	85.07	0.60	5.88	41.61
C++						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
CodeT5+	**95.53**	**95.46**	**99.06**	**47.97**	**85.36**	**97.93**
CodeBERT	92.33	92.48	97.68	27.99	68.68	94.66
PolyCoder	67.31	67.83	73.53	0.40	6.40	40.06

TABLE 8. The influence of integrated metrics to the performance of MageCode. (Unit: %).

Python						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
Metric Integrated	**98.46**	**98.70**	**99.83**	**86.87**	96.14	**99.77**
CodeT5+ Only	98.41	98.65	99.82	86.37	**97.26**	99.75
Java						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
Metric Integrated	**98.05**	**98.59**	**99.47**	**47.89**	**85.45**	**99.76**
CodeT5+ Only	97.98	98.54	99.43	44.09	84.02	99.69
C++						
	Accuracy	F1 Score	AUROC	TPR@0.1%FPR	TPR@1%FPR	TPR@10%FPR
Metric Integrated	**95.53**	**95.46**	**99.06**	**47.97**	85.36	97.93
CodeT5+ Only	95.28	95.31	99.02	38.56	**86.99**	**98.07**

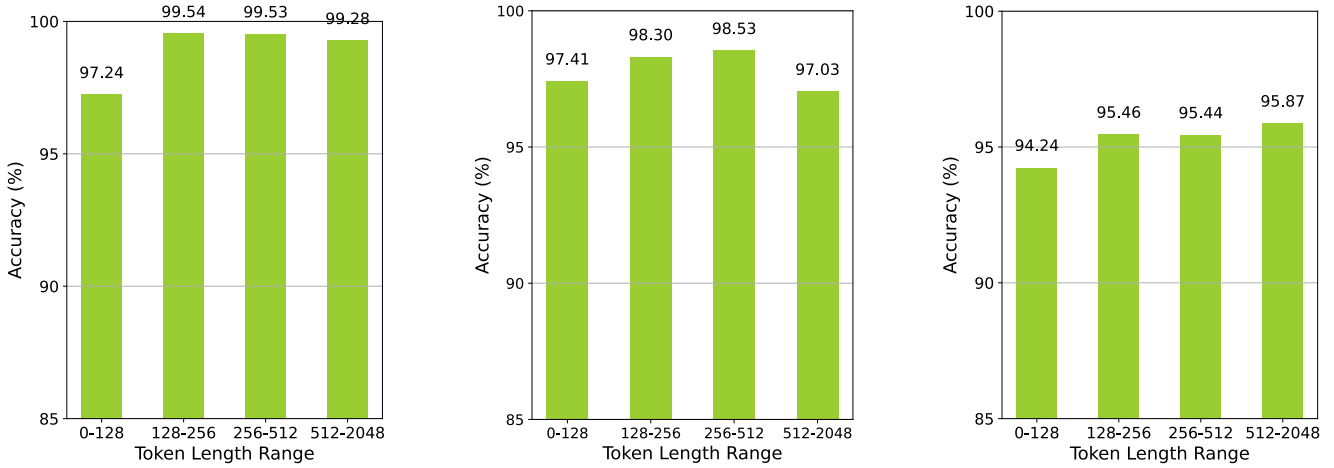
F1-Score, AUROC, and TPR@0.1%FPR across three datasets. The disparity in performance between these two scenarios is minor when applied to Python and Java datasets. Specifically, in the Python dataset, *Metric Integrated* outperforms *CodeT5+ Only* by 0.05% in Accuracy, 0.05% in F1-Score, and 0.01% in AUROC. The Java dataset has values of 0.07%, 0.05%, and 0.04%, respectively. The superior performance of *Metric Integrated* over *CodeT5+ Only* is particularly evident in the C++ dataset, exhibiting a 0.15% increase in Accuracy, 0.15% increase in F1-Score, and 0.04% increase in AUROC. Furthermore, while *CodeT5+ Only* may exhibit superior performance in TPR@1%FPR (for Python and C++) and TPR@10%FPR (for C++), Metric Integrated consistently surpasses it in TPR@0.1%FPR, with enhancements of 0.5%, 3.8%, and 9.41% in Python, Java, and C++, respectively.

Nevertheless, the notable performance of *CodeT5+ Only* underscores the exceptional competence of the CodeT5+ model in comprehending and analyzing source code, which significantly contributes to MageCode. Moreover, the improvement in the performance of MageCode when integrating various metrics into the feature vector has demonstrated the beneficial impact of these metrics on the method, which is consistent with the evaluation findings in Section V-A.

3) INFLUENCE OF TOKENS LENGTH TO DETECTION METRICS

Since MageCode is limited to processing a maximum of 512 tokens, this subsection further analyzes the accuracy score of MageCode for each token length range. This subsection divided the original test set into smaller test sets, grouping source code with the same token length range into one group (e.g., 0-128, 128-256, etc.). The results are shown in Figure 8.

We clearly observe that the model achieves the lowest accuracy score when the input source code is too short (below 128 tokens). This observation aligns with some previous work on developing machine-generated text detectors [31]. As the



(**a**) Python (**b**) Java (**c**) C++

FIGURE 8. **The influence of token length to the performance of MageCode on three different datasets.**

token length increases (128-256, 256-512, etc.), the accuracy also gradually increases. In the case of Python and Java, the method achieves the highest accuracy when the token length range is within 128-256 or 256-512. These two ranges exhibit a competitive level of accuracy, with a variation of 0.01% in Python. This finding also holds true for C++ with a decrease of 0.02%. Regarding Java, the difference is further pronounced, increasing by 0.23% from 98.30% to 98.53%. However, this difference is relatively minor when compared to the disparities in other ranges. When the input token length exceeds 512, we observe a slight drop in accuracy. This outcome is understandable given that long source code may include significant features at the end that the model does not process due to the truncation mechanism. C++ is the only special case where the truncation of lengthy tokens does not negatively affect the detection accuracy but conversely results in the highest accuracy across all test subsets.

This section has presented evaluation results from three main experiments. First, the applicability of six metric-based machine-generated text detection methods on source code input has been explored. Four of them, namely Log-Likelihood, Log-Rank, Entropy, and GLTR, show promising results and are integrated into the MageCode method. The remaining two are excluded due to the poor performance of LRR and inferior results of Rank in comparison to its analogous metrics - Log-Rank.

Second, the performance of the MageCode method has been evaluated and compared to existing baselines. The experiment results demonstrate superior performance of MageCode, attaining an accuracy over 98% for Python and Java, and 95% for C++, with a true positive rate surpassing 85% while maintaining a false positive rate lower than 1%.

Finally, the section concludes with ablation studies examining the impact of various factors on MageCode performance: pre-trained LLM, integrated metrics, and token length of source code input. In MageCode, three pre-trained LLMs are taken into account: CodeT5+, CodeBERT, and PolyCoder. CodeT5+ and CodeBERT both exhibit superior and comparable results, while PolyCoder suffers from poor performance. Besides, the evaluation of MageCode performance on two scenarios (one with integrated metrics and the other without using these metrics) demonstrates the beneficial impact of those metrics on the detection ability of the MageCode method. The last experiments focus on token length by dividing the test sets into groups based on varying token length ranges and assessing the accuracy of MageCode for each group. Groups of 128-256 and 256-512 tokens show competitive and highest accuracies since the source code in these groups is sufficiently lengthy to avoid the truncation mechanism, except for the C++ case, whose accuracy is highest when evaluating on the 512-2048 token length range group.

VI. CONCLUSION

This paper presents MageCode, a novel machine-generated code detection method. MageCode utilizes the pre-trained encoder-only CodeT5+ 220M model to extract features from source code input. This method also makes use of various metric-based machine-generated text detection methods for performance improvement. The evaluation results demonstrate the superior performance of MageCode compared to existing baselines, with high accuracy and true positive rate while maintaining false positive rate lower than 1%.

This paper has explored the applicability of several metric-based machine-generated text detection methods to source code analysis. Log-Likelihood, Log-Rank, Entropy, and GLTR were proven to be beneficial and integrated into MageCode, which make a positive impact on the performance of the method. The effect of other factors, such as pre-trained LLM and token length, on the effectiveness

of MageCode is also considered. In considered scenarios, MageCode outperforms benchmarks (OpenAI Detector and DetectGPT4Code), achieving up to 98.46% accuracy for Python. In MageCode, incorporating the pre-trained model CodeT5+ with metric-based techniques enhances performance compared to utilizing only the pre-trained model CodeT5+. Besides, source code with different token lengths also has a considerable effect on the performance of MageCode, where the method tends to achieve higher accuracy when being applied to source code with token lengths in the ranges of 128-256 and 256-512.

To conduct experiments, we constructed a new dataset for the machine-generated code detection problem. The dataset includes over 45,000 source code generated by three advanced large language models in code generation: GPT-4-Turbo, Gemini-pro-1.0, and Code-bison-32k. It also contains over 80,000 human-written solutions collected from previous studies and carefully preprocessed. All source code in the dataset are written in three popular programming languages in educational environments: Python, Java, and C++.

Initially, MageCode concentrates on identifying source codes produced by humans or LLMs. In the future, we will enhance MageCode to detect code similarity across various source code files and to discover flaws inside these files. Furthermore, MageCode emphasized the educational environment; nevertheless, this research can be expanded to identify unethical applications of AI in job applications, coding competitions, and similar contexts.

REFERENCES

[1] S. R. Das and M. J. V., "Perceptions of higher education students towards ChatGPT usage," *Int. J. Technol. Educ.*, vol. 7, no. 1, pp. 86–106, Feb. 2024.

[2] P. Haindl and G. Weinberger, "Students' experiences of using ChatGPT in an undergraduate programming course," *IEEE Access*, vol. 12, pp. 43519–43529, 2024.

[3] X. He, X. Shen, Z. Chen, M. Backes, and Y. Zhang, "MGTBench: Benchmarking machine-generated text detection," 2023, *arXiv:2303.14822*.

[4] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, "DetectGPT: Zero-shot machine-generated text detection using probability curvature," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2023, pp. 24950–24962.

[5] Z. Xu and V. S. Sheng, "Detecting AI-generated code assignments using perplexity of large language models," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 21, pp. 23155–23162.

[6] Y. Wang, H. Le, A. Deepak Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "CodeT5+: Open code large language models for code understanding and generation," 2023, *arXiv:2305.07922*.

[7] OpenAI et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[8] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.

[9] Google. (2024). *Code Bison Repository*. Accessed: Jun. 18, 2024. [Online]. Available: https://console.cloud.google.com/vertex-ai/publishers/google/model-garden/code-bison

[10] (2024). *Code Models Overview*. Accessed: Jun. 21, 2024. [Online]. Available: https://cloud.google.com/vertex-ai/generative-ai/docs/code/code-models-overview

[11] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.

[12] S. J. Prince, *Understanding Deep Learning*. Cambridge, MA, USA: MIT Press, 2023.

[13] I. Solaiman, M. Brundage, J. Clark, A. Askell, A. Herbert-Voss, J. Wu, A. Radford, G. Krueger, J. Wook Kim, S. Kreps, M. McCain, A. Newhouse, J. Blazakis, K. McGuffie, and J. Wang, "Release strategies and the social impacts of language models," 2019, *arXiv:1908.09203*.

[14] S. Gehrmann, H. Strobelt, and A. M. Rush, "GLTR: Statistical detection and visualization of generated text," 2019, *arXiv:1906.04043*.

[15] J. Su, T. Yue Zhuo, D. Wang, and P. Nakov, "DetectLLM: Leveraging log rank information for zero-shot detection of machine-generated text," 2023, *arXiv:2306.05540*.

[16] T. GLM et al., "ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools," 2024, *arXiv:2406.12793*.

[17] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P. Wendell, M. Zaharia, and R. Xin. (2023). *Free Dolly: Introducing the World's First Truly Open Instruction-tuned Llm*. [Online]. Available: https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm

[18] Y. Anand, Z. Nussbaum, A. Treat, A. Miller, R. Guo, B. Schmidt, G. Community, B. Duderstadt, and A. Mulyar, "GPT4All: An ecosystem of open source compressed language models," 2023, *arXiv:2311.04931*.

[19] J. Tow. *Stablelm Alpha V2 Models*. Accessed: Jun. 1, 2023. [Online]. Available: https://huggingface.co/stabilityai/stablelm-base-alpha-7b-v2

[20] *Claude*. Accessed: Jun. 1, 2023. [Online]. Available: https://claude.ai/

[21] B. Guo, X. Zhang, Z. Wang, M. Jiang, J. Nie, Y. Ding, J. Yue, and Y. Wu, "How close is ChatGPT to human experts? Comparison corpus, evaluation, and detection," 2023, *arXiv:2301.07597*.

[22] X. Yang, K. Zhang, H. Chen, L. Petzold, W. Yang Wang, and W. Cheng, "Zero-shot detection of machine-generated codes," 2023, *arXiv:2310.05103*.

[23] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[24] R. Li, J. Fu, B.-W. Zhang, T. Huang, Z. Sun, C. Lyu, G. Liu, Z. Jin, and G. Li, "TACO: Topics in algorithmic COde generation dataset," 2023, *arXiv:2312.14852*.

[25] Y. Li et al., "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, Dec. 2022.

[26] H. Z. Dosilovic and I. Mekterovic, "Robust and scalable online code execution system," in *Proc. 43rd Int. Conv. Inf., Commun. Electron. Technol. (MIPRO)*, Sep. 2020, pp. 1627–1632.

[27] D. Ippolito, D. Duckworth, C. Callison-Burch, and D. Eck, "Automatic detection of generated text is easiest when humans are fooled," 2019, *arXiv:1911.00650*.

[28] V. M. Hanriot, L. C. B. Torres, and A. P. Braga, "Multiclass graph-based large margin classifiers: Unified approach for support vectors and neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 3, no. 1, pp. 1–10, Apr. 2024.

[29] T. Akshar, V. Singh, N. L. B. Murthy, A. Krishna, and L. Kumar, "A codebert based empirical framework for evaluating classification-enabled vulnerability prediction models," in *Proc. 17th Innov. Softw. Eng. Conf.*, Feb. 2024, pp. 1–11, doi: 10.1145/3641399.3641405.

[30] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proc. 6th ACM SIGPLAN Int. Symp. Mach. Program.*, Jun. 2022, pp. 1–10.

[31] V. Verma, E. Fleisig, N. Tomlin, and D. Klein, "Ghostbuster: Detecting text ghostwritten by large language models," 2023, *arXiv:2305.15047*.



HUNG PHAM received the bachelor's degree in computer engineering from the School of Information and Communications Technology, Hanoi University of Science and Technology, in 2023, where he is currently the master's degree. He is working for the Back Khoa Cyber Security (BKCS) Center. His research interests include cybersecurity and trusted computing.



HUYEN HA received the bachelor's degree in computer science from the School of Information and Communications Technology, Hanoi University of Science and Technology, in 2024. She is currently with the Back Khoa Cyber Security (BKCS) Center and the School of Information and Communications Technology, Hanoi University of Science and Technology. Her research interests include cybersecurity and large language models.



VAN TONG received the B.S. degree in information technology from the Talented Information and Communication Technology Program, Hanoi University of Science and Technology, in 2017, and the Ph.D. degree in information technology from the University of Paris-Est Créteil, in 2021. He is currently a Lecturer with the School of Information and Communication Technology, Hanoi University of Science and Technology. His research interests include blockchain, cybersecurity, SDN, and network troubleshooting. He has served on conference and workshop program committees for IEEE INFOCOM 2019 and IEEE HPSR 2021. Besides, he has reviewed for top international conferences and journals.



DUNG HOANG received the B.S. degree in computer engineering from Hanoi University of Science and Technology, Hanoi, Vietnam, in 2023, where he is currently pursuing the M.S. degree. In 2022, he was a Research Assistant with the Bach Khoa Cyber Security Center, School of Information and Technology, Hanoi University of Science and Technology. His research interests include computer networking and information security, focusing on anomaly detection in networking.



DUC TRAN received the M.Sc. degree in research and the Ph.D. degree in computer science from City University London, in 2013 and 2015, respectively. He is currently a Lecturer with the School of Information and Communication Technology, Hanoi University of Science and Technology, and the Director of the Bach Khoa Cyber Security Center (BKCS). His current research interests include machine learning, pattern recognition, and computer security. His current and past application areas of his work include biometric authentication, network security, and multimedia security. He served on the program committee for the SOICT 2016 and SOIS 2018, 2022 conferences.



TUYEN NGOC LE received the B.S. degree in mathematics and information from Hanoi National University of Education, Hanoi, Vietnam, in 1996, the M.S. degree in computer science from Le Quy Don Technical University, Hanoi, in 2006, and the Ph.D. degree in electronic engineering from the National Kaohsiung University of Science and Technology, Taiwan, in 2015. He is currently an Assistant Professor with the Department of Electronic Engineering, Ming Chi University of Technology, Taipei, Taiwan. His research interests include network security, digital image processing, biometric technologies, such as face recognition, fingerprint classification, hand gestures, medical image processing, and artificial intelligence-enabled automated optical inspection systems.